

Week 9 Project:

**Modernizing a Legacy Inventory Management Service: A Java and Object-Oriented
Redesign**

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

August 28, 2026

The inventory management service has run since 2008 on a single Linux virtual machine, and it works. The case for replacing it is not that it is old but that it can no longer reach the numbers the business requires. NFR-1 asks for 500 transactions per second at a 95th-percentile latency below 75 ms; the service delivers roughly 220 tx/s at 115 ms, and the overview names the cause — a single global lock in `order.c`. This proposal analyzes why the current design cannot close that gap, selects a paradigm and language that can, and states what the change costs.

Part One: Legacy System Analysis

Abstraction and Encapsulation in the Current System

The codebase is modular but not encapsulated, and that distinction explains much of its present difficulty. Header and source separation gives `inventory.h` a published surface — `add_item`, `remove_item` — while implementation stays in the `.c` file, which is genuine abstraction at file granularity. Encapsulation fails one level down. The struct-plus-function pattern pairs struct `Item` with an `init_item` helper, but C has no private fields, so any caller may reach past the helper and write a quantity directly; the invariant the helper exists to protect is advisory rather than enforced. Module-level static arrays hold stock counts as shared mutable state, coupling modules that never call one another and defeating unit testing, since no test can construct an isolated instance. Errors travel as return codes the compiler never obliges a caller to inspect, and the overview reports the predictable result: omissions are common.

Evaluating C 99 Against the Four Criteria

Readability suffers less from C itself than from a decade of incremental patching; mixed standards, heavy pointer arithmetic, and minimal commenting place the comprehension cost on

whoever inherits a file. Maintainability is the weakest criterion: manual memory management and global variables interact badly, and with few automated tests the intermittent segmentation faults are hard to reproduce, let alone correct. Performance is genuinely strong in the small — 48 ms median latency, 120 MB peak RAM, a quarter-second cold start. In the large it is bounded by something else: steady CPU sits at 35% with peaks near 75% across four vCPUs, so the machine is not saturated when throughput plateaus, and the ceiling is serialization rather than instruction speed. Ecosystem support is where the language's age tells: the toolchain is mature, but the JSON and REST libraries a networked service now assumes are absent, so capability the team should inherit must instead be built.

The Type System and Its Implications

C 99 is statically but weakly typed, a pairing that carries much of the cost of each with only part of the benefit. Static checking catches many errors before the program runs, though it also means a change to a widely included header forces full recompilation, a usability cost paid on every edit-test cycle. Weakness then undercuts the guarantee: implicit conversions are permitted, so a pointer may be cast to an integer and back without complaint, and the compiler accepts code whose behavior the standard leaves undefined (ISO/IEC, 1999). Memory safety sits outside the type system entirely — allocation is manual, arrays are unchecked, and no type prevents use after free. The implication is specific: C's types describe the shape of data but say nothing about its lifetime or range, so the defects that actually take this service down are precisely the ones its types cannot model.

Part Two: System Requirements and Paradigm Selection

Requirements to Retain and to Improve

FR-1 through FR-4 are retained unchanged. Stock adjustment, order fulfilment with rollback, the nightly reconciliation CSV, and real-time shortfall alerting are the business itself, and altering them would produce a different product. The non-functional requirements divide. NFR-1 and NFR-2 are the improvement targets and the justification for the project: 500 tx/s at p95 under 75 ms against an observed 220 tx/s at 115 ms, and horizontal scaling on commodity instances against a design confined to one virtual machine. NFR-3, a feature lead time under one sprint, is also unmet, but for a different reason — accumulated technical debt rather than throughput. NFR-4 and NFR-5 must be preserved and are met largely at the deployment tier.

Suitability of the Candidate Paradigms

Three paradigms were weighed against those requirements rather than in the abstract. Procedural programming, the status quo, is not disqualified by speed; C is fast. It is disqualified by the absence of any construct binding an invariant to the data it protects, so guarding stock counts falls back on a lock the programmer must remember to take, which is how one global lock came to gate the entire system. Functional programming attacks that directly, since immutable data has no contended write and pure transformations parallelize freely, and it suits FR-3, a report computed over a snapshot. It suits FR-2 poorly, because reserving stock is inherently a mutation of shared state, and expressing it functionally demands machinery the team must learn before shipping anything. Object-oriented programming is the recommendation: it lets an InventoryItem own its quantity and enforce its own non-negativity invariant, converting a global discipline into a local one and making per-item locking expressible.

Key Programming Styles Applied to This System

Four styles bear directly on this system. Modularity already exists at file granularity but without enforced boundaries; in Java, packages and access modifiers turn a boundary violation into a compile error rather than a review comment. Side effects are the crux: the legacy design shares mutable stock counts through static arrays, and every such share is a place the global lock must be held, so narrowing the scope of mutation is what permits the lock to be split.

Immutability applies selectively: Sku, Quantity, and Reservation are immutable value types, while the stock level itself must change and is therefore confined to the single object that guards it. Inheritance is used sparingly, since it couples a subclass to its parent's implementation; the redesign prefers interfaces for repositories and alert channels, because these requirements call for substitutable implementations rather than shared hierarchy (Bloch, 2018).

Solving the Core Problem: Order Fulfilment

The core problem is FR-2 under the global lock. The legacy process_order takes one lock covering all inventory, so two orders for entirely unrelated SKUs serialize against each other and concurrency is capped however many of the sixteen threads sit idle. The object-oriented reformulation gives each item its own lock and its own invariant.

```

CLASS InventoryItem
  sku      : immutable
  quantity : private, mutable
  lock     : private, one lock per item
  METHOD reserve(amount) RETURNS Reservation
    ACQUIRE lock
    TRY
      IF amount > quantity THEN
        PUBLISH StockShortfallAlert(sku, amount, quantity) // FR-4
        THROW InsufficientStockException(sku)
        quantity <- quantity - amount
        RETURN new Reservation(sku, amount)
      FINALLY
        RELEASE lock
    METHOD release(reservation) // rollback path
      ACQUIRE lock
      TRY    quantity <- quantity + reservation.amount
      FINALLY RELEASE lock

CLASS OrderService
  METHOD fulfil(order)
    held <- empty list
    TRY
      FOR EACH line IN order.lines SORTED BY line.sku // deadlock-free
        ordering
          item <- repository.find(line.sku)
          held.add( item.reserve(line.quantity) )
          repository.commit(held)
    CATCH InsufficientStockException
      FOR EACH r IN held DO repository.find(r.sku).release(r) // FR-2 rollback
      RETHROW

```

Two properties do the work. Locking is per SKU, so unrelated orders no longer wait on one another, and acquiring in sorted SKU order makes deadlock unreachable for multi-line orders. Rollback is structural rather than remembered: the catch block releases exactly the reservations it took, so the partial-failure path cannot be skipped the way an ignored return code can.

Conceptual System Outline

The modular structure is four layers, each depending only on the one beneath it. An API layer terminates TLS and enforces role-based access (NFR-5). A service layer holds transaction boundaries, of which OrderService is one member. A domain layer holds InventoryItem, Reservation, and Sku with the invariants they protect. A repository layer owns persistence behind an interface, retiring the SQL embedded in C files by giving every query one home (Fowler, 2002). Flow control follows a request down and back: the API validates and authorizes,

the service opens a transaction and calls the domain, the domain enforces its invariants and throws when one would be violated, and the repository persists. Abstraction is by interface at every seam, so the reconciliation job (FR-3) reads through the same repository the request path uses.

Comparing the Original and Selected Paradigms

Table 1
Procedural and Object-Oriented Paradigms Compared for This System

Dimension	Procedural (C 99, current)	Object-oriented (Java, proposed)
Maintainability	Invariants held by convention; static arrays couple modules; few tests, because state cannot be isolated	Invariants enforced by the compiler through private state; dependencies injected via interfaces, so units are testable alone
Team learning curve	Familiar to current maintainers, but the knowledge is tacit, undocumented, and concentrated in long-tenured staff	Widely taught and hired for, lowering onboarding cost, though the team must learn the JVM memory and concurrency model
Problem-solving effectiveness	Direct control of memory and layout, but no construct binds an invariant to data, so concurrency control stays global and manual	Objects localize state together with the lock that guards it, which is what makes per-SKU concurrency expressible

Table 1 is not uniformly favorable and should not be read that way. The paradigm change buys enforcement and testability and pays for them in low-level control. For a service whose ceiling is contention rather than instruction throughput that trade is worth making, and the learning-curve row is the one to manage deliberately during transition.

Part Three: Language Recommendation and Redesign

Evaluating Java Against Python

Table 2
Java and Python Evaluated Against the System's Needs

Criterion	Java	Python
Type system	Static and strong; whole classes of error are rejected at compile time	Dynamic and strong; type errors surface at runtime unless annotations and a checker are adopted
Concurrency	Genuine multicore parallelism; virtual threads make per-request concurrency inexpensive	The global interpreter lock serializes bytecode execution, so parallelism

		needs separate processes (Gross, 2023)
Throughput	JIT-compiled and within a small factor of C for long-running services	Interpreted, and roughly an order of magnitude slower per core on CPU-bound work (Pereira et al., 2021)
Ecosystem fit	Mature server frameworks, pooled JDBC drivers, and strong production observability	Strongest in data science and machine learning; capable but lighter for this transactional workload
Operational profile	Higher baseline heap and a warm-up period before peak performance	Lower memory floor, but an interpreter and full dependency tree on every host

Java is the recommendation, and NFR-1 decides it. The requirement is 500 transactions per second at p95 below 75 ms from a workload that is concurrent by nature. CPython permits only one thread to execute bytecode at a time, so a Python implementation reaches that figure by running many processes, each carrying its own cache and connection pool — an operational tax compounding with every instance added (Gross, 2023) — on a runtime already an order of magnitude slower per core on CPU-bound work (Pereira et al., 2021). Java obtains parallelism from the runtime it already has. The second reason is the type system: Part One identified weak typing and unchecked error returns as the source of this system's most damaging defects, and replacing a weakly typed language with a dynamically typed one would move those failures from compile time into production rather than remove them.

Supporting the Company's Long-Term Goals

Three company goals follow. Scalability improves because the redesign holds no request state above the domain layer, so instances scale horizontally on commodity hardware as NFR-2 requires, and per-SKU locking means added cores do useful work instead of queuing behind one mutex. Maintainability improves against NFR-3, which asks for a feature lead time under two weeks; the current codebase misses that target because changes are risky rather than large, and compiler-enforced encapsulation with units testable in isolation reduces the cost of a change

more than any feature could. Development efficiency improves through the ecosystem the overview identifies as missing: a mature build toolchain, a deep hiring pool, and a standard library already covering JSON, HTTP, and connection pooling.

Control Flow and Exception Handling

The legacy `process_order` signals failure with integer return codes the compiler never requires a caller to inspect, and it carries a distinct unlock-failure path of its own. The Java equivalent moves that logic into the object that owns the data, making failure unignorable and cleanup automatic.

```
public final class InventoryItem {  
    private final Sku sku;  
    private int quantity;           // private: no caller writes it  
    directly  
    private final Lock lock = new ReentrantLock(); // one lock per item, not one global  
    lock  
  
    public Reservation reserve(int amount, AlertChannel alerts)  
        throws InsufficientStockException, OrderException {  
  
        if (amount <= 0) {  
            throw new IllegalArgumentException("amount must be positive");  
        }  
        lock.lock();  
        try {  
            if (amount > quantity) {  
                alerts.publish(new StockAlert(sku, amount, quantity)); // FR-4  
                throw new InsufficientStockException(sku);  
            }  
            quantity -= amount;  
            return new Reservation(sku, amount);  
        } catch (AlertDeliveryException e) {  
            throw new OrderException("alerting failed for " + sku, e);  
        } finally {  
            lock.unlock();           // released on every path, normal or exceptional  
        }  
    }  
}
```

Three differences are structural rather than stylistic. A caller cannot quietly ignore a checked `InsufficientStockException` the way it can ignore a non-zero int; the compiler requires it be handled or declared. The finally block releases the lock on every exit path, including exceptional ones, which removes the legacy `ERR_UNLOCK_FAIL` case entirely. And chaining

the original exception as a cause means an on-call engineer reads a trace naming the real fault rather than an error code to be looked up in a header.

Regular Expressions for SKU Recognition

The legacy has `_valid_sku` compiles its pattern on every call and returns only yes or no. The same grammar supports considerably more once the pattern is compiled once and its parts are named.

```
public final class Sku {  
    private static final Pattern PATTERN =  
        Pattern.compile("(?<dept>[A-Z]{3})-(?<serial>[0-9]{4})$");  
  
    private final String dept;  
    private final int serial;  
  
    public static Sku parse(String raw) {  
        Matcher m = PATTERN.matcher(raw.strip());  
        if (!m.matches()) { // recognition failure  
            throw new InvalidSkuException(raw);  
        }  
        return new Sku(m.group("dept"), Integer.parseInt(m.group("serial")));  
    }  
}
```

Hoisting `Pattern.compile` into a static field pays the compilation cost once for the life of the process rather than once per validated SKU, which is not a micro-optimization at 500 transactions per second. Named capture groups let the one expression that recognizes a SKU also destructure it, making recognition and parsing a single step; and because `parse` either returns a valid `Sku` or throws, a malformed identifier cannot travel deeper into the system disguised as an ordinary string.

The Redesigned Architecture

The legacy architecture was one C server on one virtual machine: a command-line client over a raw TCP socket, sixteen fixed POSIX threads, an in-process hash-table cache, and the flat modules inventory, order, report, and util reaching MySQL through inline ODBC calls. Figure 1 shows what replaces it. Four changes carry the proposal: the single server becomes stateless

instances behind a load balancer, the flat modules become four layers with enforced boundaries, the scattered ODBC calls collapse into a repository, and the one global lock becomes per-SKU striping.

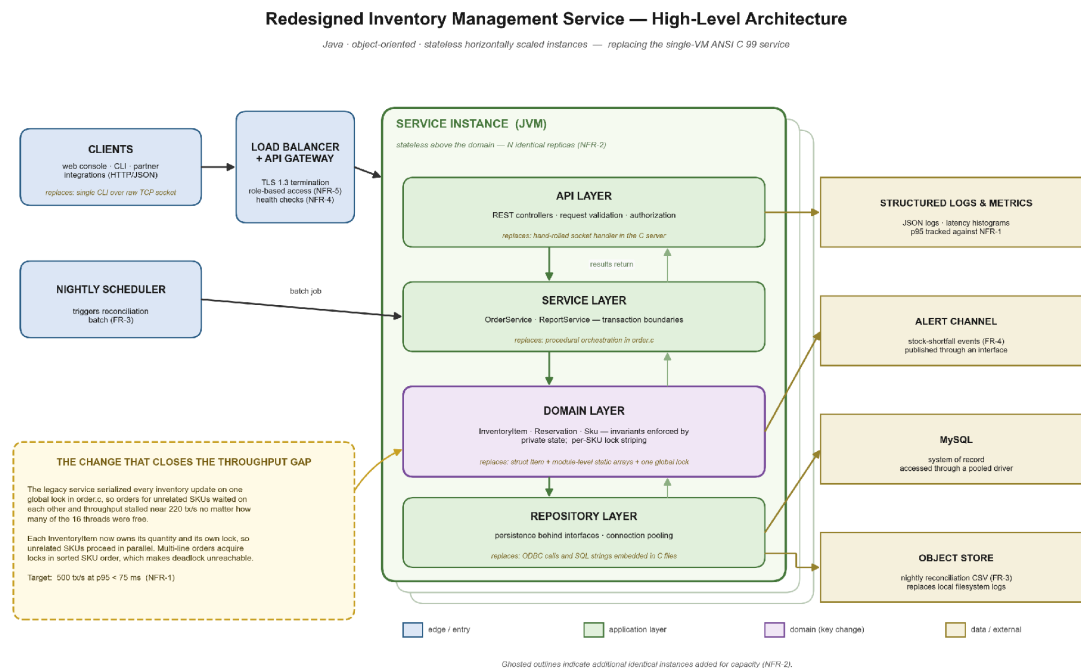


Figure 1. High-level architecture of the redesigned inventory management service. *Italicized notes on each layer identify the legacy component it replaces.*

Tradeoffs Considered

Four costs deserve statement rather than omission. Performance in the small gets worse before it gets better: the JVM starts more slowly than the legacy's 0.25 s cold start and its heap will exceed the observed 120 MB peak, so the redesign wins on throughput while losing on footprint. The sharpest risk is garbage collection measured against NFR-1's 75 ms tail, since a stop-the-world pause lands squarely in the 95th percentile; a low-pause collector such as ZGC, with disciplined allocation on the hot path, is therefore part of the recommendation rather than a tuning exercise deferred to later (Oracle, n.d.). Extensibility improves, since a new alert channel is another implementation of an existing interface, though layering adds indirection that makes

small changes touch more files. Tooling improves decisively. Team skills are the genuine risk: the maintainers are C programmers whose tacit knowledge of this domain is the asset most easily destroyed by a rewrite, which argues for strangling the legacy service one module at a time behind the new API rather than attempting a cutover (Fowler, 2019).

Conclusion

The recommendation is an object-oriented redesign in Java, deployed as stateless instances behind a load balancer, with per-SKU locking replacing the global lock in `order.c`. It is justified by a measured gap — 220 tx/s at p95 115 ms against a requirement of 500 tx/s at 75 ms — and by a diagnosis identifying contention rather than instruction speed as the cause. The costs are a larger memory footprint, a collector that must be chosen deliberately, and a team that needs time to retrain. Each is manageable; the throughput ceiling, on the current architecture, is not.

References

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Fowler, M. (2019). Strangler fig application.

<https://martinfowler.com/bliki/StranglerFigApplication.html>

Gross, S. (2023). PEP 703 – Making the global interpreter lock optional in CPython. Python Software Foundation. <https://peps.python.org/pep-0703/>

ISO/IEC. (1999). Information technology — Programming languages — C (ISO/IEC 9899:1999).

Oracle. (n.d.). The Z garbage collector. In HotSpot virtual machine garbage collection tuning guide. <https://docs.oracle.com/en/java/javase/21/gctuning/>

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2021). Ranking programming languages by energy efficiency. Science of Computer Programming, 205, 102609. <https://doi.org/10.1016/j.scico.2021.102609>

Note on generative AI use: Generative AI (Anthropic's Claude) was used to assist with structuring, and editing this document. All design decisions, technical claims, and conclusions were conceived by the author.